

AWS & KUBERNETES DEEP DIVE SERIES

AWS Application Load Balancer Ingress on Amazon EKS

From subnet tagging to Helm install — every component explained with the why, the what, and the how everything connects.

Covers: ALB fundamentals • IRSA • IAM policies • Helm install • Ingress YAML deep dive

DevOps Shack | @devopsshack | 2026

Chapter 1: What Is ALB Ingress and Why Do We Need It?

Before running a single command, you need to understand what problem we are solving. This is what makes every subsequent step feel logical rather than mechanical.

1.1 The Problem — How Does Outside Traffic Reach Your Pods?

When you deploy an app on Kubernetes (EKS), your Pods are running inside the cluster on private IP addresses — completely unreachable from the internet. You need something to sit at the edge and forward incoming HTTP/HTTPS requests to the right pods.

Kubernetes has three options for exposing services to the world:

Option	What It Does	When to Use
NodePort	Exposes a port on every worker node's IP. Works but is ugly — users need to know node IPs and a random high port.	Local dev / testing only
LoadBalancer (Service)	Creates one cloud load balancer per Service. If you have 10 services, you pay for 10 load balancers.	Simple single-service apps
Ingress	One load balancer handles ALL services. Routes traffic by hostname or path. Cost-effective and feature-rich.	Production multi-service apps

The Ingress advantage in one sentence:

Instead of one load balancer per service (expensive, unmanageable), Ingress gives you one intelligent layer-7 load balancer that routes all traffic by URL path or hostname — like a smart traffic cop at the entrance of your cluster.

1.2 What Is an Ingress Controller?

Kubernetes defines the Ingress resource as a specification — but it does NOT implement it. You need to install an Ingress Controller, which is a pod running inside your cluster that watches for Ingress resources and actually configures a real load balancer.

Think of it like this:

- Ingress resource = the traffic routing rules you write in YAML (e.g. /api goes to backend, / goes to frontend)
- Ingress Controller = the software that reads those rules and programs a real load balancer to enforce them

There are many ingress controllers — Nginx, HAProxy, Traefik, Kong. On AWS EKS, the best choice is the AWS Load Balancer Controller, which creates native AWS Application Load Balancers (ALBs).

1.3 What Is an ALB and Why Is It Better Than Nginx for AWS?

Feature	Nginx Ingress	AWS ALB Ingress
Load balancer type	Software LB running as a pod	Native AWS ALB — managed by AWS
TLS termination	Done in the pod (uses cluster CPU)	Done at the ALB (free of charge to pod)
Health checks	Kubernetes liveness probes	ALB target group health checks (more robust)
WAF integration	Manual setup	Native AWS WAF integration
Cost	Pod resources (compute)	ALB pricing — efficient at scale
AWS Certificate Manager	Manual cert mounting	Native ACM certificate attachment
Auth/cognito	Requires custom config	Native Cognito integration

1.4 The Full Architecture — How All Pieces Connect

Here is the complete picture before we start building. Each numbered component corresponds to a step in this guide:

1. Internet user sends HTTP/HTTPS request to a domain name (e.g. app.devopsshack.com).
2. DNS resolves the domain to an AWS Application Load Balancer DNS name.
3. The ALB receives the request and checks its listener rules.
4. The ALB forwards the request to an EKS worker node's NodePort based on target group routing.
5. The kube-proxy on that node routes the packet to the correct Pod.
6. The Pod processes the request and returns a response — back through the same path.

Behind the scenes, who creates and manages the ALB? The AWS Load Balancer Controller pod running inside kube-system. It watches your Ingress YAML and automatically provisions, configures, and updates the ALB. When you delete the Ingress, it deletes the ALB too.

The connection chain you are building:

Your Ingress YAML → Controller reads it → Controller calls AWS APIs → ALB created → Target groups attached to nodes → Traffic flows.

The controller needs IAM permissions to call AWS APIs. That permission is granted via IRSA (IAM Roles for Service Accounts) — which is what Steps 3, 4, and 5 set up.

Chapter 2: Pre-Checks — Making Sure the Foundation Is Solid

Rushing into ALB setup on a broken cluster is the fastest way to spend hours debugging symptoms that have nothing to do with ALB. These pre-checks take 2 minutes and save hours.

2.1 Step 0 — Verify Cluster Connectivity and Node Health

The cluster must be healthy before you add any new components. A node that is NotReady will not receive ALB target group traffic, even if everything else is configured correctly.

```
# Check that all nodes are Ready
kubectl get nodes

# Expected output:
NAME                                                    STATUS    ROLES    AGE
ip-10-0-1-100.ap-south-1.compute.internal             Ready     <none>    3d
ip-10-0-2-200.ap-south-1.compute.internal             Ready     <none>    3d

# Check all system pods are running (nothing in Error or CrashLoopBackOff)
kubectl get pods -A

# Focus on kube-system - these must all be Running/Completed:
kubectl get pods -n kube-system
```

What to fix if nodes are NotReady:

Check the node's EC2 instance health in the AWS console. Common causes: instance stopped, security group blocking kubelet port 10250, or the node group IAM role missing required policies (AmazonEKSWorkerNodePolicy, AmazonEKS_CNI_Policy, AmazonEC2ContainerRegistryReadOnly).

2.2 Step 1 — Remove Old HAProxy Ingress (If Present)

If you previously installed HAProxy or Nginx as your Ingress Controller, you must remove the old Ingress resource for your app BEFORE installing the ALB controller. Here is exactly why.

Why you cannot have two Ingress Controllers fighting over the same Ingress resource

An Ingress resource has an annotation called `kubernetes.io/ingress.class` or uses the `ingressClassName` field. If you have both HAProxy and ALB controllers running, and your Ingress resource does not explicitly specify which controller to use, BOTH controllers will try to own it.

This causes:

- Duplicate load balancers being created (wasted cost)
- HAProxy's routing rules conflicting with ALB's routing rules

- Unpredictable traffic behaviour — some requests hit HAProxy, others hit ALB
- Both controllers fighting to update the Ingress resource's status field

```
# Delete the old HAProxy-managed Ingress for your app
kubectl delete ingress app-ingress -n project

# Verify it is gone
kubectl get ingress -n project

# — OPTIONAL: Full HAProxy cleanup —————
# Only if you want to completely remove HAProxy from the cluster:
helm uninstall haproxy-ingress -n kube-system

# Verify HAProxy pods are gone:
kubectl get pods -n kube-system | grep haproxy
```

Why is Step 1 optional for some setups?

If you never installed HAProxy (fresh cluster), skip this step entirely. If you have other apps still using HAProxy, delete **ONLY** the Ingress resource for the app you are migrating to ALB — do not uninstall the HAProxy controller until all apps have migrated.

Chapter 3: Subnet Tagging — How the ALB Finds Your Network

This is one of the most commonly skipped steps, and it causes one of the most frustrating errors: the ALB controller reconciles your Ingress, calls AWS APIs to create the ALB — and fails with 'no subnets found'. Your entire cluster can be running fine, but without these tags, ALB simply cannot discover which subnets to deploy into.

3.1 Why Does ALB Need Subnet Discovery?

An ALB is a regional AWS resource that must be placed in specific subnets. Unlike an EC2 instance (where you choose the subnet manually), the ALB controller creates the ALB automatically — it needs to know which subnets to use WITHOUT you hard-coding them.

The controller discovers subnets by looking for AWS resource tags on your VPC subnets. This is called subnet auto-discovery.

3.2 The Two Types of Tags and What They Mean

Tag Key	Tag Value	Meaning	When to Apply
<code>kubernetes.io/role/elb</code>	1	Public subnet — place internet-facing ALBs here	Always set on public subnets
<code>kubernetes.io/role/internal-elb</code>	1	Private subnet — place internal ALBs here	Set on private subnets if using internal ALBs
<code>kubernetes.io/cluster/<cluster-name></code>	owned or shared	Marks which subnets belong to this EKS cluster	Set automatically by eksctl — verify it exists

For our use case (internet-facing ALB), we need `kubernetes.io/role/elb = 1` on every public subnet that the ALB should span. An ALB must span at least 2 Availability Zones for high availability.

3.3 Checking and Adding Subnet Tags

```
# List all subnets in your region with their tags
aws ec2 describe-subnets --region ap-south-1

# More targeted: find subnets already tagged for ELB
aws ec2 describe-subnets \
  --region ap-south-1 \
  --filters "Name=tag:kubernetes.io/role/elb,Values=1"
```

```
# If no subnets come back, you need to tag them.
# First find your public subnet IDs:
aws ec2 describe-subnets \
  --region ap-south-1 \
  --filters "Name=tag:Name,Values=*public*" \
  --query
"Subnets[].{ID:SubnetId,AZ:AvailabilityZone,Name:Tags[?Key=='Name']|[0].Value}" \
  --output table

# Tag each public subnet (replace subnet-XXXXXXX with your actual IDs):
aws ec2 create-tags \
  --resources subnet-0abc1234 subnet-0def5678 \
  --tags Key=kubernetes.io/role/elb,Value=1

# Also ensure the cluster ownership tag is present:
aws ec2 create-tags \
  --resources subnet-0abc1234 subnet-0def5678 \
  --tags Key=kubernetes.io/cluster/my-cluster,Value=shared
```

Common mistake — tagging private subnets instead of public:

Public subnets are the ones with a route to an Internet Gateway (IGW) in their route table. Private subnets route to a NAT Gateway instead. Tag the WRONG subnets and the ALB will be placed in private subnets — completely unreachable from the internet even though DNS resolves fine. Always double-check the route table before tagging.

3.4 Verifying Tags Are Correct

```
# Confirm the tag exists on your subnets
aws ec2 describe-subnets \
  --region ap-south-1 \
  --filters "Name=tag:kubernetes.io/role/elb,Values=1" \
  --query "Subnets[].{SubnetId:SubnetId, AZ:AvailabilityZone, CIDR:CidrBlock}" \
  --output table

# Expected output (at least 2 subnets in different AZs):
-----
| DescribeSubnets |
+-----+
| AZ | CIDR | SubnetId |
+-----+
| ap-south-1a | 10.0.1.0/24 | subnet-0abc1234 |
| ap-south-1b | 10.0.2.0/24 | subnet-0def5678 |
+-----+
```

Chapter 4: IAM Policy — Giving the Controller Permission to Call AWS APIs

The AWS Load Balancer Controller is a Kubernetes pod running inside your cluster. When it needs to create, update, or delete an ALB, it must call AWS APIs — just like you do with the AWS CLI. For that, it needs AWS credentials and the right IAM permissions.

This chapter explains why we create a specific IAM policy, what that policy contains, and why it is scoped the way it is.

4.1 What APIs Does the Controller Need to Call?

The controller performs a lot of AWS operations on your behalf. Here is a summary of what it does and which AWS services it must access:

AWS Service	What the Controller Does	Example API Calls
Elastic Load Balancing (ALB)	Creates/updates/deletes ALBs, listeners, listener rules, target groups	CreateLoadBalancer, ModifyRule, DeleteTargetGroup
EC2	Describes VPCs, subnets, security groups. Creates/manages security groups for the ALB	DescribeSubnets, CreateSecurityGroup, AuthorizeSecurityGroupIngress
Certificate Manager (ACM)	Lists and describes TLS certificates for HTTPS listeners	ListCertificates, DescribeCertificate
WAF v2	Associates WAF web ACLs with ALBs (if configured)	AssociateWebACL, GetWebACL
Shield	Associates Shield Advanced protection (if configured)	GetSubscriptionState
IAM	Lists server certificates for HTTPS	GetServerCertificate

Rather than manually writing all these permissions, the AWS Load Balancer Controller project maintains an official IAM policy JSON file on GitHub that covers exactly what is needed — no more, no less.

4.2 Downloading and Creating the IAM Policy

```
# Download the official policy JSON (Windows PowerShell):
Invoke-WebRequest `
  https://raw.githubusercontent.com/kubernetes-sigs/aws-load-balancer-
  controller/v2.14.1/docs/install/iam_policy.json `
  -OutFile iam_policy.json

# On Linux/Mac:
curl -o iam_policy.json \
```



```
https://raw.githubusercontent.com/kubernetes-sigs/aws-load-balancer-
controller/v2.14.1/docs/install/iam_policy.json

# — OPTIONAL: Delete if a stale version already exists —
aws iam delete-policy \
  --policy-arn arn:aws:iam::585593375370:policy/AWSLoadBalancerControllerIAMPolicy

# Create the policy in your AWS account:
aws iam create-policy \
  --policy-name AWSLoadBalancerControllerIAMPolicy \
  --policy-document file://iam_policy.json

# Note the returned PolicyArn — you need it in Step 5:
# arn:aws:iam::585593375370:policy/AWSLoadBalancerControllerIAMPolicy
```

Why download from GitHub instead of writing manually?

The policy has over 80 individual permissions across 6 AWS services. Writing it manually is error-prone and will go stale as new controller versions require additional permissions. Always use the official policy JSON from the exact controller version you are installing (v2.14.1 in our case).

4.3 Why the Policy Uses Conditions (Least Privilege)

You might look at the policy and notice some statements have Condition blocks like this:

```
# Example from the policy — the controller can only create security groups
# that are tagged as owned by the load balancer controller:
{
  "Effect": "Allow",
  "Action": "ec2:CreateSecurityGroup",
  "Resource": "*",
  "Condition": {
    "StringEquals": {
      "aws:RequestTag/elbv2.k8s.aws/cluster": "true"
    }
  }
}
```

These conditions implement least-privilege: the controller can only create or modify AWS resources that it itself tagged when creating. It cannot arbitrarily delete any security group in your account — only ones it previously created. This is defence-in-depth.

Chapter 5: IRSA — The Bridge Between Kubernetes Pods and AWS IAM

This is marked as 'very critical' in the original guide, and it deserves a full explanation. IRSA (IAM Roles for Service Accounts) is the mechanism that lets a specific Kubernetes pod assume a specific AWS IAM role — without any hardcoded AWS credentials inside the pod.

5.1 The Problem IRSA Solves

Your Load Balancer Controller pod needs to call AWS APIs. It needs AWS credentials. The old (bad) ways of solving this:

Approach	How It Works	Why It Is Bad
Hardcode in deployment YAML	AWS_ACCESS_KEY_ID and SECRET in environment variables	If YAML leaks, credentials leak. Manual rotation. No audit trail.
Use the node's IAM role	EC2 instance profile gives all pods on that node the same permissions	ALL pods on ALL nodes get those permissions. One compromised pod = full AWS access.
IRSA (the right way)	Each pod gets unique, scoped IAM permissions via its ServiceAccount	Pod-level isolation. No static credentials. Auto-rotating. Auditable.

5.2 How IRSA Works Mechanically

IRSA uses OIDC (OpenID Connect) — the same standard used in our GitHub Actions guide. Here is the flow for a pod:

7. The pod's ServiceAccount is annotated with an IAM role ARN.
8. When the pod starts, the EKS control plane injects a projected token into the pod at `/var/run/secrets/eks.amazonaws.com/serviceaccount/token`.
9. The AWS SDK inside the pod (used by the controller) finds this token automatically.
10. The SDK calls STS `AssumeRoleWithWebIdentity` using the token.
11. STS verifies the token against the cluster's OIDC provider.
12. STS issues temporary credentials valid for 1 hour — auto-refreshed by the SDK.
13. The controller uses those credentials to call ELB, EC2, and other APIs.

The key insight:

The pod never sees a long-lived AWS access key. It sees a Kubernetes-signed JWT that gets exchanged for 1-hour AWS credentials automatically. If the pod is deleted, the token goes with it. Nothing to rotate, nothing to leak.

5.3 Step 4 — Associate the OIDC Provider

Before IRSA can work, you must register your EKS cluster's built-in OIDC provider with AWS IAM. Each EKS cluster has a unique OIDC issuer URL (you can see it in the AWS console under the cluster's Details tab).

```
# Associate the cluster's OIDC provider with IAM
# This is a one-time operation per cluster
eksctl utils associate-iam-oidc-provider \
  --region ap-south-1 \
  --cluster my-cluster \
  --approve

# Verify the OIDC provider was created in IAM:
aws iam list-open-id-connect-providers

# Expected output includes your cluster's OIDC URL:
# arn:aws:iam::585593375370:oidc-provider/oidc.eks.ap-south-1.amazonaws.com/id/ABCDEF1234567890
```

What does --approve do?

eksctl shows you a summary of changes it will make and asks for confirmation. --approve skips the prompt and applies immediately. The change is: registering the cluster's OIDC issuer URL as a trusted identity provider in your AWS account's IAM. This is safe and reversible.

Chapter 6: Binding the IAM Role to the Kubernetes Service Account

Now that the OIDC provider is registered, you need to create three things in one command: a Kubernetes ServiceAccount, an IAM Role, and a trust policy that links them together. This is what lets the controller pod assume the IAM role.

6.1 What Gets Created and Why Each Piece Exists

Resource Created	Lives In	Purpose
Kubernetes ServiceAccount	kube-system namespace	Identity for the controller pod to run as inside Kubernetes
IAM Role	AWS IAM	AWS identity that the ServiceAccount can temporarily become
IAM Trust Policy	Attached to the IAM Role	Specifies: only the controller's ServiceAccount in this cluster can assume this role — nothing else

6.2 The Command and What Every Flag Means

```
eksctl create iamserviceaccount \
  --cluster=my-cluster \
  --namespace=kube-system \
  --name=aws-load-balancer-controller \
  --attach-policy-arn=arn:aws:iam::585593375370:policy/AWSLoadBalancerControllerIAMPolicy \
  --override-existing-serviceaccounts \
  --region ap-south-1 \
  --approve
```

Flag	Value	Why This Value
--cluster	my-cluster	The EKS cluster where the ServiceAccount will live and whose OIDC provider will be trusted
--namespace	kube-system	The controller runs in kube-system. ServiceAccount must be in the same namespace as the pod.
--name	aws-load-balancer-controller	This name must match exactly what the Helm chart expects. The chart references this ServiceAccount by name.

<code>--attach-policy-arn</code>	<code>...AWSLoadBalancerControllerIAMPolicy</code>	The policy from Step 3. This attaches it to the auto-created IAM role.
<code>--override-existing-serviceaccounts</code>	(flag, no value)	If you ran this before and the SA already exists, update it instead of failing. Safe to include always.
<code>--approve</code>	(flag, no value)	Skip the confirmation prompt. <code>eksctl</code> would show you a plan and ask 'proceed?' — this says yes.

6.3 What the Generated Trust Policy Looks Like

`eksctl` automatically creates an IAM role with a trust policy scoped to your specific cluster and service account. This is the security gate — it ensures only the controller in your cluster can assume this role:

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": {
      "Federated": "arn:aws:iam::585593375370:oidc-provider/oidc.eks.ap-south-1.amazonaws.com/id/ABCDEF"
    },
    "Action": "sts:AssumeRoleWithWebIdentity",
    "Condition": {
      "StringEquals": {
        "oidc.eks.ap-south-1.amazonaws.com/id/ABCDEF:sub":
          "system:serviceaccount:kube-system:aws-load-balancer-controller",
        "oidc.eks.ap-south-1.amazonaws.com/id/ABCDEF:aud": "sts.amazonaws.com"
      }
    }
  }]
}
```

The sub condition translates to: 'Only the service account named `aws-load-balancer-controller` in the `kube-system` namespace of this specific cluster can assume this role.' Any other pod, any other service account, any other cluster — rejected.

6.4 Verifying What Was Created

```
# Verify the Kubernetes ServiceAccount exists with the role annotation:
kubectl get serviceaccount aws-load-balancer-controller -n kube-system -o yaml

# Look for this annotation in the output:
```

```
# annotations:
#   eks.amazonaws.com/role-arn: arn:aws:iam::585593375370:role/eksctl-my-cluster-
#   ...

# Verify the IAM role was created:
aws iam list-roles --query "Roles[?contains(RoleName,'load-balancer')].RoleName"
```

Chapter 7: Installing the AWS Load Balancer Controller via Helm

With IAM permissions sorted, you now install the actual controller software into your cluster. Helm is the package manager for Kubernetes — think of it like apt or yum for your cluster.

7.1 What Is Helm and Why Use It Here?

The AWS Load Balancer Controller consists of multiple Kubernetes resources: a Deployment, a Service, two ValidatingWebhookConfigurations, CRDs (Custom Resource Definitions), RBAC roles and bindings, and more. Writing all this YAML manually would be hundreds of lines and error-prone.

Helm packages all of this into a Chart — a versioned, templated bundle. You install the Chart with a few `--set` flags to configure it for your cluster, and Helm handles the rest.

7.2 The Install Commands Explained

```
# Step 1: Add the AWS EKS Helm chart repository
helm repo add eks https://aws.github.io/eks-charts

# Step 2: Update the local chart cache
helm repo update eks

# Step 3: Install the controller
helm install aws-load-balancer-controller \
  eks/aws-load-balancer-controller \
  -n kube-system \
  --set clusterName=my-cluster \
  --set serviceAccount.create=false \
  --set serviceAccount.name=aws-load-balancer-controller \
  --version 1.14.0
```

Helm Flag	Value	Why This Matters
helm install <release-name>	aws-load-balancer-controller	The Helm release name. Used for upgrades: <code>helm upgrade aws-load-balancer-controller ...</code>
eks/aws-load-balancer-controller	(chart name)	The chart from the EKS repo we just added. Helm downloads this and installs it.
-n kube-system	(namespace)	Installs all resources into kube-system. Must match where the ServiceAccount lives.
--set clusterName=my-cluster	my-cluster	The controller uses this to tag AWS resources and to filter which Ingress resources to watch.

<code>--set serviceAccount.create=false</code>	false	CRITICAL: Tells Helm NOT to create a new ServiceAccount. We already created one with IRSA annotation in Step 5. If Helm creates its own, the IRSA annotation is missing and credentials fail.
<code>--set serviceAccount.name=aws-load-balancer-controller</code>	(must match Step 5)	Tell Helm which ServiceAccount to use. Must exactly match the <code>--name</code> from the <code>eksctl</code> command.
<code>--version 1.14.0</code>	Helm chart version	Pin the chart version for reproducibility. Helm chart v1.14.0 corresponds to controller image v2.14.x.

The most common mistake at this step:

Forgetting `--set serviceAccount.create=false`. Helm creates a new ServiceAccount without the `eks.amazonaws.com/role-arn` annotation. The controller pod starts but all AWS API calls fail with 'no identity-based policy allows the `sts:AssumeRoleWithWebIdentity` action'. Always use `serviceAccount.create=false` when you pre-created the SA with `eksctl`.

7.3 Step 7 — Verifying the Controller Is Running

```
# Check the Deployment is available (READY should show 2/2):
kubectl get deployment -n kube-system aws-load-balancer-controller

# Expected output:
NAME                                READY    UP-TO-DATE    AVAILABLE    AGE
aws-load-balancer-controller        2/2      2              2            3m

# Check the pods are Running (not CrashLoopBackOff or Error):
kubectl get pods -n kube-system | grep aws-load-balancer-controller

# Expected output (two replicas for HA):
aws-load-balancer-controller-6d4b5f7c8d-abc12    1/1    Running    0    3m
aws-load-balancer-controller-6d4b5f7c8d-xyz34    1/1    Running    0    3m

# Check controller logs to confirm it connected to AWS successfully:
kubectl logs -n kube-system -l app.kubernetes.io/name=aws-load-balancer-controller | tail -30
```

What healthy logs look like:

You should see lines like: 'Successfully acquired leader lease', 'Starting workers', and crucially NO lines containing 'AccessDenied' or 'no identity-based policy'. If you see AccessDenied, the IRSA setup from Steps 4-5 needs debugging.

Chapter 8: Writing the Ingress YAML — Telling the ALB What to Do

Now that the controller is running, creating an ALB is as simple as applying an Ingress resource. The controller watches for Ingress resources with the alb ingress class and automatically provisions the ALB in AWS.

8.1 A Complete Production-Ready Ingress YAML

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: app-ingress
  namespace: project
  annotations:
    # Tell the controller this Ingress belongs to ALB:
    kubernetes.io/ingress.class: alb

    # Internet-facing ALB (use 'internal' for private ALBs):
    alb.ingress.kubernetes.io/scheme: internet-facing

    # Route to pods directly (IP mode) — recommended for EKS:
    alb.ingress.kubernetes.io/target-type: ip

    # HTTPS: attach an ACM certificate (replace with your cert ARN):
    # alb.ingress.kubernetes.io/certificate-arn: arn:aws:acm:ap-south-
    # 1:585593375370:certificate/xxx

    # Health check settings for target groups:
    alb.ingress.kubernetes.io/healthcheck-path: /health
    alb.ingress.kubernetes.io/healthcheck-interval-seconds: '15'

spec:
  ingressClassName: alb
  rules:
    - host: app.devopsshack.com          # optional: remove for any hostname
      http:
        paths:
          - path: /api
            pathType: Prefix
            backend:
              service:
                name: backend-service
                port:
                  number: 8080
          - path: /
            pathType: Prefix
            backend:
              service:
                name: frontend-service
                port:
                  number: 80
```

8.2 Key Annotations Explained

Annotation	Value	What It Does
<code>alb.ingress.kubernetes.io/scheme</code>	<code>internet-facing</code>	Creates a public ALB with public IP addresses. Use 'internal' for a VPC-only ALB.
<code>alb.ingress.kubernetes.io/target-type</code>	<code>ip</code>	Routes traffic directly to Pod IPs (bypasses NodePort). Requires VPC CNI — standard on EKS. Preferred over 'instance' mode.
<code>alb.ingress.kubernetes.io/certificate-arn</code>	ACM cert ARN	Attaches a TLS certificate. ALB terminates HTTPS, sends HTTP to pods — free TLS offloading.
<code>alb.ingress.kubernetes.io/healthcheck-path</code>	<code>/health</code>	The URL path ALB uses to health check pods. Must return HTTP 200 for the target to be considered healthy.
<code>kubernetes.io/ingress.class</code>	<code>alb</code>	Tells the ALB controller this Ingress is for it. The Nginx or HAProxy controller ignores Ingresses with this class.

8.3 Applying the Ingress and Watching the ALB Get Created

```
# Apply the Ingress resource:
kubectl apply -f ingress.yaml

# Watch the controller provision the ALB (takes 1-3 minutes):
kubectl get ingress app-ingress -n project -w

# Once ready, the ADDRESS column shows the ALB DNS name:
NAME          CLASS   HOSTS          ADDRESS
PORTS   AGE
app-ingress   alb     app.devopsshack.com  k8s-project-appingre-xxx.ap-south-
1.elb.amazonaws.com  80      2m

# You can also see the ALB in the AWS console:
# EC2 > Load Balancers > filter by 'k8s-project-'

# Test the ALB directly (before DNS is set up):
curl http://k8s-project-appingre-xxx.ap-south-1.elb.amazonaws.com/
curl http://k8s-project-appingre-xxx.ap-south-1.elb.amazonaws.com/api
```

What happens behind the scenes when you kubectl apply the Ingress:

1) Controller detects new Ingress with class 'alb'. 2) Reads annotations and spec. 3) Calls EC2 APIs to describe subnets with elb tag. 4) Calls ELB APIs to create ALB in those subnets. 5) Creates a listener on port 80 (and 443 if cert is set). 6) Creates listener rules for each path. 7) Creates a target group for each backend service. 8) Registers pod IPs (ip mode) or node ports (instance mode) in the target groups. 9) Updates the Ingress resource's status.loadBalancer.ingress with the ALB DNS.

Chapter 9: Troubleshooting — What Goes Wrong and How to Fix It

Most ALB setup failures fall into a small number of categories. This chapter covers the most common ones with root cause, diagnosis, and fix.

9.1 Ingress ADDRESS Is Empty After 5+ Minutes

The ALB was never created. The controller either did not see the Ingress or failed to provision the ALB.

```
# Step 1: Check if the controller is running:
kubectl get pods -n kube-system | grep aws-load-balancer-controller

# Step 2: Read the controller logs for errors:
kubectl logs -n kube-system \
  -l app.kubernetes.io/name=aws-load-balancer-controller \
  --tail=50

# Common log errors and their meaning:
# 'no subnets found'          -> subnets missing kubernetes.io/role/elb tag (fix:
Chapter 3)
# 'AccessDenied'              -> IRSA not working — IAM role not assumed (fix:
Chapter 5-6)
# 'not found: ingress class' -> ingressClassName not matching (fix: check
annotation)
# 'certificate not found'     -> ACM cert ARN wrong or in wrong region
```

9.2 Controller Pod in CrashLoopBackOff

```
# Get detailed error from the crashing pod:
kubectl describe pod -n kube-system <pod-name>
kubectl logs -n kube-system <pod-name> --previous

# Most common cause: wrong ServiceAccount or missing annotation
kubectl get sa aws-load-balancer-controller -n kube-system -o yaml

# The annotation must be present:
# annotations:
#   eks.amazonaws.com/role-arn: arn:aws:iam::ACCOUNT:role/ROLE_NAME

# If annotation is missing, the eksctl step failed.
# Fix: re-run Chapter 6 Step 5 eksctl command
```

9.3 ALB Created But Returns 503

The ALB exists and receives traffic, but the target group has no healthy targets.

```
# Find the target group in AWS console:
# EC2 > Target Groups > filter by cluster name

# Check target health via CLI:
aws elbv2 describe-target-health \
  --target-group-arn arn:aws:elasticloadbalancing:ap-south-
1:585593375370:targetgroup/k8s-project-xxx/yyy

# Common reasons for unhealthy targets:
# 1. Health check path returning non-200 (fix: set correct healthcheck-path
annotation)
# 2. Security group blocking ALB -> Pod traffic (fix: allow ALB SG in pod's node
SG)
# 3. Pod not ready (fix: kubectl get pods -n project and check pod status)
```

Chapter 10: Quick Reference — All Steps in Order

Use this as your cheat sheet. Every command in the correct sequence.

Step 0 — Pre-check

```
kubectl get nodes
kubectl get pods -A
```

Step 1 — Remove old HAProxy Ingress (if present)

```
kubectl delete ingress app-ingress -n project
# Optional full cleanup:
helm uninstall haproxy-ingress -n kube-system
```

Step 2 — Tag public subnets for ALB discovery

```
aws ec2 create-tags \
  --resources subnet-XXXXXXXXX subnet-YYYYYYYYY \
  --tags Key=kubernetes.io/role/elb,Value=1

# Verify:
aws ec2 describe-subnets --region ap-south-1 --filters
"Name=tag:kubernetes.io/role/elb,Values=1"
```

Step 3 — Create IAM policy

```
curl -o iam_policy.json https://raw.githubusercontent.com/kubernetes-sigs/aws-load-balancer-controller/v2.14.1/docs/install/iam_policy.json
aws iam create-policy --policy-name AWSLoadBalancerControllerIAMPolicy --policy-document file://iam_policy.json
```

Step 4 — Associate OIDC provider (IRSA prerequisite)

```
eksctl utils associate-iam-oidc-provider --region ap-south-1 --cluster my-cluster --approve
```

Step 5 — Create ServiceAccount + IAM role binding

```
eksctl create iamserviceaccount \
  --cluster=my-cluster \
  --namespace=kube-system \
  --name=aws-load-balancer-controller \
  --attach-policy-arn=arn:aws:iam::585593375370:policy/AWSLoadBalancerControllerIAMPolicy \
  --override-existing-serviceaccounts \
```

```
--region ap-south-1 \  
--approve
```

Step 6 — Install controller via Helm

```
helm repo add eks https://aws.github.io/eks-charts  
helm repo update eks  
helm install aws-load-balancer-controller eks/aws-load-balancer-controller \  
-n kube-system \  
--set clusterName=my-cluster \  
--set serviceAccount.create=false \  
--set serviceAccount.name=aws-load-balancer-controller \  
--version 1.14.0
```

Step 7 — Verify

```
kubectl get deployment -n kube-system aws-load-balancer-controller  
kubectl get pods -n kube-system | grep aws-load-balancer-controller  
kubectl logs -n kube-system -l app.kubernetes.io/name=aws-load-balancer-controller  
| tail -20
```

Step 8 — Apply your Ingress

```
kubectl apply -f ingress.yaml  
kubectl get ingress app-ingress -n project -w  
# Wait for ADDRESS field to populate with ALB DNS name
```